

TEMA 4: PRUEBAS DE INTEGRACIÓN DE LA CAPA MODELO

INTRODUCCIÓN

- Se utilizó JUnit 5 para las pruebas de integración del servicio **MovieService**
- Pruebas de integración (NO de unidad)
 - Prueban el correcto funcionamiento de **MovieService** (**MovieServiceImpl**)
 - Prueba indirectamente el correcto funcionamiento de los DAOs dentro de MovieService que acceden a la BD (prueba la parte interna de MovieService)
- Pruebas de integración de **MovieService** → residen en **MovieServiceTest**
 - Implementadas tradicionalmente
 - Convención de nombrado → **XxxServiceTest**
 - Diseñados varios casos de prueba para cada caso de uso → cada uno implementado en un método **@Test**
 - Conveniente probar + de 1 caso de prueba en un único método **@Test**

REPASO

- JUnit 5 consta de 3 módulos
 - JUnit Platform → módulo core
 - **JUnit Jupiter** → nuevo modelo de programación de pruebas
 - JUnit Vintage → permite ejecutar pruebas escritas con JUnit 3 o JUnit 4
- Implementación de 3 casos de prueba para el caso de uso **buyMovie**

```
// ...
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
// ...

public class MovieServiceTest {

    private static MovieService movieService = null;
    private static SqlSaleDao saleDao = null;

    private Movie getValidMovie(String title) {
        return new Movie(title, (short) 85, "Movie description",
            19.95F);
    }

    private Movie getValidMovie() {
        return getValidMovie("Movie title");
    }
}
```

```
private Movie createMovie(Movie movie) {

    Movie addedMovie = null;
    try {
        addedMovie = movieService.addMovie(movie);
    } catch (InputValidationException e) {
        throw new RuntimeException(e);
    }
    return addedMovie;
}

private void removeMovie(Long movieId) {

    try {
        movieService.removeMovie(movieId);
    } catch (InstanceNotFoundException |
        MovieNotRemovableException e) {
        throw new RuntimeException(e);
    }
}
```

```
private void removeSale(Long saleId) {

    DataSource dataSource = DataSourceLocator
        .getDataSource(MOVIE_DATA_SOURCE);

    try (Connection connection = dataSource.getConnection()) {

        try {

            /* Prepare connection. */
            connection.setTransactionIsolation(
                Connection.TRANSACTION_SERIALIZABLE);
            connection.setAutoCommit(false);

            /* Do work. */
            saleDao.remove(connection, saleId);

            /* Commit. */
            connection.commit();

        } catch (InstanceNotFoundException e) {
            connection.commit();
            throw new RuntimeException(e);
        } catch (SQLException e) {
            connection.rollback();
            throw new RuntimeException(e);
        } catch (RuntimeException|Error e) {
            connection.rollback();
            throw e;
        }

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

}
```

```

--▶ org.junit.jupiter.api.Test
@Test
public void testBuyMovieAndFindSale()
    throws InstanceNotFoundException, InputValidationException,
    SaleExpirationException {

    Movie movie = createMovie(getValidMovie());
    Sale sale = null;

    try {

        // Buy movie
        LocalDateTime beforeBuyDate =
            LocalDateTime.now().withNano(0);

        sale = movieService.buyMovie(movie.getMovieId(), USER_ID,
            VALID_CREDIT_CARD_NUMBER);

        LocalDateTime afterBuyDate =
            LocalDateTime.now().withNano(0);

        // Find sale
        Sale foundSale = movieService.findSale(sale.getSaleId());

        // Check sale
        assertEquals(sale, foundSale);
        assertEquals(VALID_CREDIT_CARD_NUMBER,
            foundSale.getCreditCardNumber());
        assertEquals(USER_ID, foundSale.getUserId());
        assertEquals(movie.getMovieId(), foundSale.getMovieId());
        assertEquals(movie.getPrice(), foundSale.getPrice());
        assertTrue(foundSale.getExpirationDate().compareTo(
            beforeBuyDate.plusDays(SALE_EXPIRATION_DAYS)) >= 0) &&
            (foundSale.getExpirationDate().compareTo(
            afterBuyDate.plusDays(SALE_EXPIRATION_DAYS)) <= 0));
        assertTrue(foundSale.getSaleDate().compareTo(
            beforeBuyDate) >= 0) &&
            (foundSale.getSaleDate().compareTo(afterBuyDate) <= 0));
        assertTrue(foundSale.getMovieUrl().startsWith(
            BASE_URL + sale.getMovieId()));

    } finally {
        // Clear database: remove sale (if created) and movie
        if (sale != null) {
            removeSale(sale.getSaleId());
        }
        removeMovie(movie.getMovieId());
    }
}

```

Los test deben dejar la BD como estaba al principio, por eso elimina la película

```

@Test
public void testBuyMovieWithInvalidCreditCard() {

    Movie movie = createMovie(getValidMovie());

    try {

        assertThrows(InputValidationException.class, () -> {

            Sale sale = movieService.buyMovie(movie.getMovieId(),
                USER_ID, INVALID_CREDIT_CARD_NUMBER);
            removeSale(sale.getSaleId());

        });

    } finally {
        // Clear database
        removeMovie(movie.getMovieId());
    }
}

```

Expresión lambda (segundo parámetro) que debería devolver la excepción especificada en el primer parámetro de **assertThrows**

```

@Test
public void testBuyNonExistentMovie() {

    assertThrows(InstanceNotFoundException.class, () -> {
        Sale sale = movieService.buyMovie(NON_EXISTENT_MOVIE_ID,
            USER_ID, VALID_CREDIT_CARD_NUMBER);
        removeSale(sale.getSaleId());
    });

}

// Casos de prueba para el resto de casos de uso...

} // class

```

ASPECTOS ESPECÍFICOS

CONCEPTOS BÁSICOS DE PRUEBAS EN MAVEN

- El código de pruebas se ubica en el directorio **src/test** (tiene una estructura de directorios similar a **src/main**)
- El código de las clases de prueba está contenido en el paquete **es.udc.ws.movies.test.model** (y subpaquetes), dentro del módulo **ws-movies-model**
- Las pruebas se pueden ejecutar automáticamente desde Maven (a través del plugin **surefire**) haciendo que se ejecute la fase **test** (ej. **mvn test**)
 - Maven genera un informe detallado en **target/surefire-reports**
 - Por defecto, se consideran clases de pruebas aquellas clases de **src/test/java** cuyo nombre empieza o termina por **Test**

INDEPENDENCIA ENTRE CASOS DE PRUEBA

- Mantenimiento del código → importante atender a que cada caso de prueba
 - [1] Cree los datos que precise
 - [2] Invoque a la operación a probar
 - [3] Realice las comprobaciones necesarias
 - [4] Elimine los datos que crearon en el paso [1] y los que se pudieron generar en el paso [2]
- ↑ Cada caso de prueba es independiente del resto de casos de prueba
 - La ejecución de un caso de prueba puede asumir que sólo existen en BD los datos que él mismo ha creado en [1]
 - Modificar, eliminar o añadir un caso de prueba no tiene incidencia en el resto de los casos
 - Esta independencia favorece que distintas personas a lo largo del tiempo puedan modificar las clases de prueba de manera “ágil”
- Los casos de prueba anteriores utilizan métodos privados → evitar replicar código
 - **createMovie**, **removeMovie** y **removeSale** → se encargan de los aspectos [1] y [4]
 - Asumen que los parámetros son correctos (porque se usan para crear datos necesarios para la ejecución de los casos de prueba [1] o para eliminar datos creados tras su ejecución [4])
 - Capturan las excepciones checked y las relanzan como **RuntimeException** → evita tener que tratar esas excepciones checked en los casos de prueba)
 - **getValidMovie** → crear cómodamente un objeto **Movie** con datos correctos

USO DE DAOS

- Los casos de prueba necesitan crear los datos necesarios [1] antes de invocar al caso de uso a probar, y finalmente eliminar los datos creados [4]
- Criterio para [1] y [4]
 - El servicio ofrece un método apropiado → se usa ese servicio (ej. **createMovie** y **removeMovie**)
 - El servicio NO ofrece un método apropiado → se usa directamente el DAO correspondiente (ej. **removeSale**)
- Cuando en un caso de prueba se realizan las comprobaciones necesarias [3], puede ser necesario hacer alguna consulta en la BD (mismo convenio que el anterior)

USO DE UNA BD ESPECÍFICA PARA EJECUTAR LAS PRUEBAS

- Buena práctica → disponer de varias BDs con el mismo esquema
 - [1] Una BD para probar la aplicación “como humano” en la fase de desarrollo
 - [2] Una BD para ejecutar las pruebas automatizadas
 - [3] Una BD para producción
- Ejs. asignatura → se dispone de BDs para [1] (**ws**) y [2] (**wstest**)
- El probar la aplicación como “humano” no altera la BD de pruebas → los casos de prueba no se encontrarán con “datos inesperados”

IGUALDAD DE OBJETOS

- **Assertions.assertEquals (Object, Object)** → compara utilizando el método **equals** (definido en **Object**)
- La implementación por defecto de **equals** en **Object** realiza una comparación por referencia
 - **sale1.equals(sale2) = true** if **sale1** y **sale2** son 2 variables que apuntan al mismo objeto **Sale**
- Comparación por contenido → necesario redefinir el método **equals**
 - Ej. **sale1.equals(sale2) = true** aunque **sale1** y **sale2** sean 2 objetos distintos pero con mismo contenido (mismo id de venta, mismo id de película, etc.) → necesario redefinir **equals**
 - La implementación de **equals** debe devolver **true** si los 2 objetos son iguales campo a campo
- Se redefine **equals** → redefinir **hashCode** de manera consistente
 - **HashCode** devuelve por defecto enteros diferentes para distintos objetos
 - Redefinir **equals** → **hashCode** debe devolver el mismo entero para objetos que, aunque distintos, son iguales de contenido
 - Esto permite usar los objetos como clave de estructuras que indexan por clave (ej. mapas)
- Ej. en **testBuyMovieAndFindSale**, para que sea posible compara **sale** y **foundSale**, es preciso redefinir **equals** (y **hashCode** por consistencia), dado que son 2 objetos distintos, y si la prueba es satisfactoria, con el mismo contenido
- Ejs. asignatura → se ha usado el IDE para genera ágilmente la implementación de **equals** y **hashCode** en **Movie** y **Sale**

INICIALIZACIÓN

- A lo largo de los casos de prueba se utilizan las variables estáticas **movieService** y **saleDao**
- Estas variables se inicializan en un método anotado con **@BeforeAll** → se ejecuta una única vez, antes de que se ejecuten los métodos anotados con **@Test**

```
@BeforeAll
public static void init() {

    // Algo más??? ...

    movieService = MovieServiceFactory.getService();
    saleDao = SqlSaleDaoFactory.getDao();

}
```

- Recordatorio tema 3
 - El constructor de la implementación de **MovieService** obtiene la referencia al **DataSource** de la siguiente manera

```
public MovieServiceImpl() {
    dataSource = DataSourceLocator.getDataSource(
        MOVIE_DATA_SOURCE);
    // ...
}
```

- [1, aplicación Web / servicio Web] Cuando la capa Modelo se ejecuta dentro de un servidor de aplicaciones, **getDataSource** tiene que localizar el **DataSource** proporcionado por el servidor de aplicaciones
- [2, pruebas capa Modelo] Cuando la capa Modelo se ejecuta desde las pruebas de integración, el desarrollador de las pruebas tiene que proporcionar un **DataSource** y el método **getDataSource** lo devolverá
- En el escenario [2], **DataSourceLocator** proporciona el método **addDataSource**, que permite registrar un **DataSource** por nombre explícitamente

```
@BeforeAll
public static void init() {

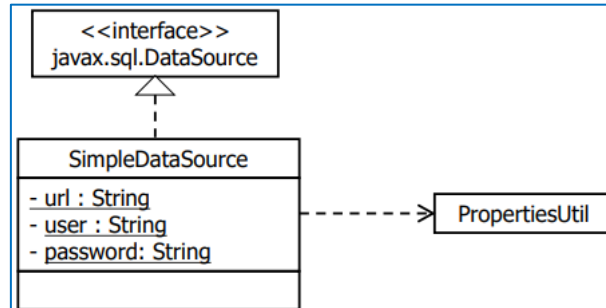
    DataSource dataSource = new SimpleDataSource();
    DataSourceLocator.addDataSource(MOVIE_DATA_SOURCE,
        dataSource);

    movieService = MovieServiceFactory.getService();
    saleDao = SqlSaleDaoFactory.getDao();

}
```


- **SimpleDataSource**

- Implementación trivial de la interfaz **DataSource** proporcionada por el módulo **ws-util**
- Implementa el método **getConnection** delegando en **DriverManager.getConnection** (para las pruebas no es necesario proporcionar pool de conexiones)
- El resto de los métodos lanzan **SQLFeatureNotSupportedException**
- Lee el valor de la URL de conexión a la BD, el usuario y la contraseña de un fichero con nombre **SimpleDataSource.properties**, que debe estar disponible en el classpath



CONFIGURACIÓN

- Maven → la configuración de las pruebas reside en **src/test/resources**
 - Cuando se ejecuta las pruebas sobre un módulo Maven, el classpath está formado → **target/test-classes**, **target/classes** y los JARs de las dependencias
- **ws-movies/ws-movies-model/src/test/resources** contiene el fichero **SimpleDataSource.properties** con la siguiente configuración
 - La URI apunta a la BD de pruebas (**wstest**)

```
SimpleDataSource.url=jdbc:mysql://localhost/wstest?...
SimpleDataSource.user=ws
SimpleDataSource.password=ws
```

DATASOURCELOCATOR

- **MovieServiceTest** debe registrar un **DataSource** explícitamente antes de poder invocar al servicio de la capa Modelo (**MovieServiceImpl**)
 - En el método **init (@BeforeAll)** de **MovieServiceTest**

```
DataSource dataSource = new SimpleDataSource();
DataSourceLocator.addDataSource(MOVIE_DATA_SOURCE,
    dataSource);
```
 - En el constructor de **MovieServiceImpl**

```
dataSource = DataSourceLocator.getDataSource(
    MOVIE_DATA_SOURCE);
```
 - El valor de la constante **MOVIE_DATA_SOURCE** es **ws-javaexamples-ds**
- Cuando la capa Modelo se ejecuta dentro de un servidor de aplicaciones, este servidor proporciona una implementación de un **DataSource** (típicamente con pool de conexiones)
- Cuando la capa Modelo se ejecute como parte de un servicio o una aplicación Web esta será la situación
- Una aplicación (app o servicio Web) instalada en un servidor de aplicaciones Java puede obtener referencias a algunos objetos (ej. **DataSources**) gestionados por el servidor de aplicaciones usando la API estándar JNDI (Java Naming and Directory Interface)
- JNDI (Java Naming and Directory Interface)
 - API (**javax.naming**) que forma parte de la API básica de Java, para localizar y registrar objetos asociándoles nombres jerárquicos
 - Los servidores de aplicaciones exponen los objetos **DataSource** gestionados por el servidor de aplicaciones mediante la API de JNDI
 - Obtención de referencias a **DataSource**

```
InitialContext initialContext = new InitialContext();
DataSource dataSource = (DataSource) initialContext.lookup(
    "java:comp/env/jdbc/ws-javaexamples-ds");
```

- **InitialContext** pertenece al paquete **javax.naming**
- Los servidores de aplicaciones tienen la obligación de exponer los objetos **DataSource** en el contexto ("directorio") **java:comp/env**
- Cuando el administrador de un servidor de aplicaciones configura un **DataSource**, por convenio, se aconseja que lo registre bajo el subcontexto **jdbc**, de manera que el nombre completo queda como **java:comp/env/jdbc/<<nombre simple del DataSource>>**
- **DataSourceLocator** mantiene un mapa de **DataSources** indexado por nombre
- **addDataSource (name, dataSource)** → añade el **DataSource** pasado como parámetro en el mapa
- **getDataSource (name)**
 - Comprueba si el **DataSource** solicitado está en el mapa
 - Está en el mapa → lo devuelve
 - No está en el mapa → el **DataSource** no fue añadido con **addDataSource** → capa Modelo corre dentro de servidor aplicaciones
 - Lo busca por JNDI
 - Lo inserta en el mapa

```
public class DataSourceLocator {

    public static final String JNDI_PREFIX = "java:comp/env/jdbc/";

    private static Map<String,DataSource> dataSources =
        Collections.synchronizedMap(new HashMap<String, DataSource>());

    private DataSourceLocator() {}

    public static void addDataSource(
        String name, DataSource dataSource) {

        dataSources.put(name, dataSource);

    }

    -----> Nombre "simple" (sin JNDI_PREFIX) del DataSource
               (e.g. ws-javaexamples-ds)
```

```
public static DataSource getDataSource(String name)
    throws RuntimeException{

    DataSource dataSource = (DataSource) dataSources.get(name);

    if (dataSource == null) {

        try {
            InitialContext initialContext = new InitialContext();
            dataSource = (DataSource) initialContext.lookup(
                JNDI_PREFIX + name);
            dataSources.put(name, dataSource);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }

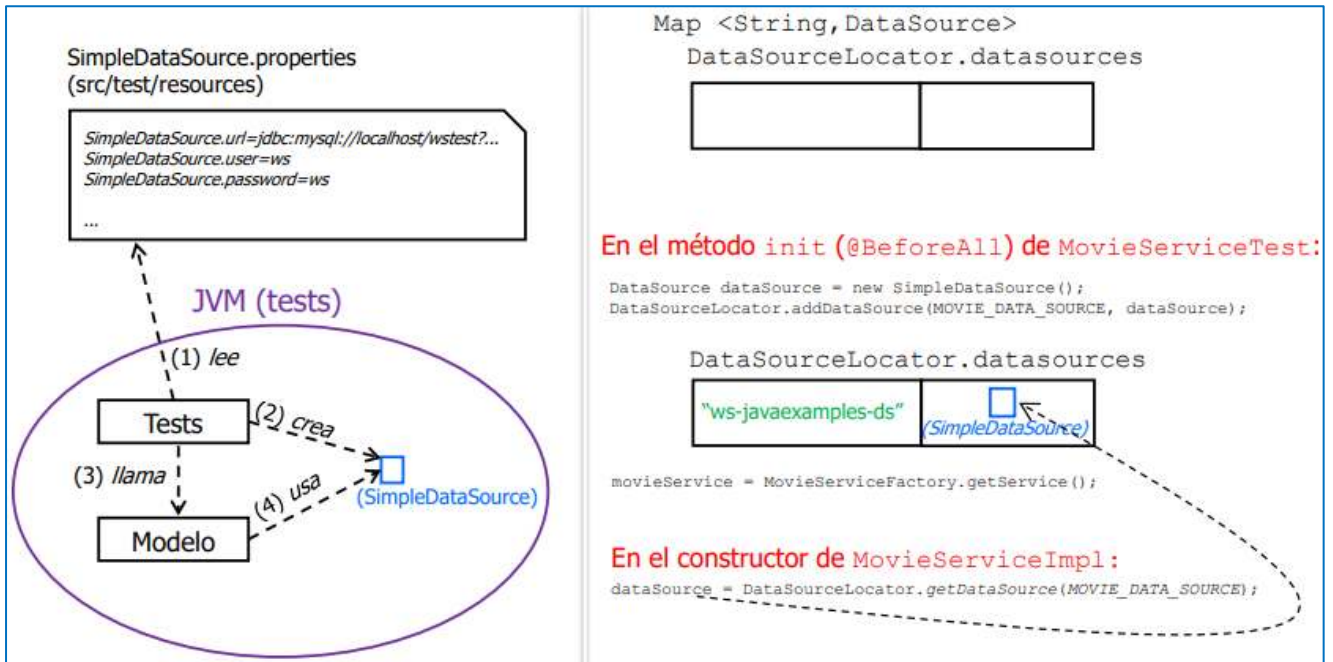
    }

    return dataSource;

}

} // class
```


DATASOURCELOCATOR – TESTS



DATASOURCELOCATOR – APLICACIÓN

